SportsQuant

A Statistical Sports-Prediction and Paper-Trading Engine
Models, Data, and the Probability Functions Being Fit

Prepared for Jesús — built with Claude (Anthropic)

September 2026

Contents

1 What this document is **2**

2 One-paragraph summary of how any of this works **2**

3 System architecture **2**

4 Data: what trains the model vs. what predicts a match **3**

5 The soccer goals model: Dixon–Coles **3**
5.1 The probability function being fit 3
5.2 Fitting: the exact objective that is maximised 4
5.3 How this was validated 4
5.4 The card / corner model 5

6 The in-play soccer engine: what actually moves the price after a goal **5**

7 Sports other than soccer **6**

8 From a scoreline table to every market on the slip **6**
8.1 Odds, edge, and staking — the definitions used throughout 7

9 What is really fitted, and what is a placeholder — read this before trusting a number **7**

10 The demo universe: why matches move even at 3am with nothing real on **7**

11 Live data and automatic refresh **8**

12 The chat **8**

13 The X / Twitter layer, and why it is off by default **8**

14 What was tested, and how **9**

15 What this replaces from the first (Antigravity) attempt, and why **9**

16 Honest limitations **9**

17 Suggested next steps **10**

1 What this document is

This is the technical reference for the sports-prediction project: which machine-learning / statistical models are used, which data trains them, which data is used to produce a prediction for a specific match, and — since the request was specifically for "which probability function are you trying to fit" — the exact functional forms, written out. It also narrates what was built, in the order it was built, including the parts of an earlier attempt (by Google's Antigravity agents) that this version replaces and why.

This is a research and paper-trading tool. Nothing in this project places a real bet, logs into a sportsbook, or should be read as financial advice. Section 9 is the most important one if you read only one: it lays out exactly which numbers in the app come from a model fitted on real data, and which come from a placeholder prior, because those are very different things and the app is designed to never blur them.

2 One-paragraph summary of how any of this works

You train a model by giving it many past examples where the outcome is already known (here: past matches, with the final score) and letting it adjust a small number of parameters — one or two numbers per team, plus a handful of shared constants — until its predicted probabilities are as consistent as possible with what actually happened, measured by a likelihood function. You then freeze those parameters and use them to compute a probability for a match whose outcome is not yet known. Every "bet" in the app is a probability read off that fitted model, converted to a fair decimal price ($1/p$), and compared against a real bookmaker price if you supply one.

3 System architecture

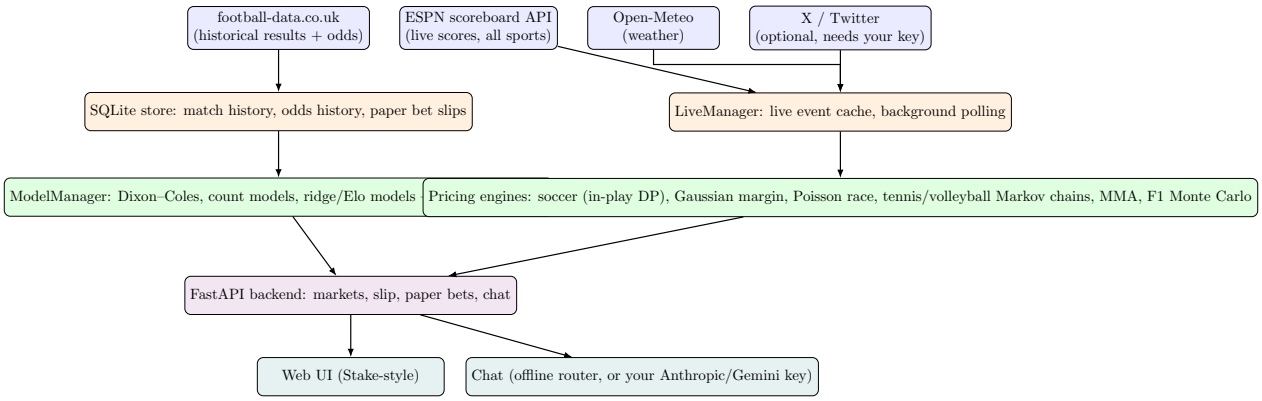


Figure 1: Data flows in from the left (real sources, top row); everything downstream of the SQLite store and the live cache is this project's own code.

Two genuinely different kinds of "AI" are at work, and it matters that they are kept separate:

- **The prediction models** (Sections 5–7) are classical statistics / maximum-likelihood models, not neural networks and not language models. They run on a CPU in milliseconds and their entire "knowledge" is a few hundred fitted numbers per league (one attack and one defence strength per team, plus a handful of shared constants).
- **The chat** (Section 12) is a large language model (your own Anthropic or Gemini API key, or a built-in zero-configuration offline keyword router if you supply no key). It is deliberately never allowed to invent a probability: every number it states comes from calling a tool that runs the actual statistical model and returns the result as data, which is then just read back in words. This is enforced in the tool-calling loop (`sportsquant/chat/engine.py`), not just requested in a prompt.

4 Data: what trains the model vs. what predicts a match

	Used to *train* (fit parameters)	**Used to *predict* one match**
Soccer (Dixon–Coles)	Full-time and half-time goals for every match in the lookback window (default 3 seasons), per league, from football-data.co.uk	The two teams' already-fitted attack-/defence numbers, home advantage, and (if unseen) a name-matching fallback to the league average
Cards / corners (count model)	Full-time cards and corners per match, with the referee's name where the source provides it (England and Scotland files; most European leagues' free files omit it)	The two teams' fitted "produces" / "concedes" rates and the named referee's fitted multiplier
Basketball / NFL (ridge ratings)	Historical box scores *once you wire up a results ingester for them* — see Section 9: today this falls back to a same-season win-rate heuristic from ESPN's own records	The two teams' rating (or, currently, their season win/loss record)
Hockey / MLB (Poisson race)	Same caveat as above	Same fallback
Tennis / volleyball	No historical serve/rally data is ingested yet	A single externally supplied win-probability estimate is inverted into a serve/rally probability (Section 7)
Live score, clock, injuries, weather	Never used for training	Pulled fresh at request time (ESPN, Open-Meteo) and fed into the in-play engine

Table 1: What is genuinely fitted today is soccer (goals, and cards/corners where the referee is recorded). Everything else has a working, honestly-labelled placeholder — see Section 9.

5 The soccer goals model: Dixon–Coles

5.1 The probability function being fit

For a match between home team i and away team j, let (X, Y) be the final home and away goal counts. The model is Dixon and Coles' (1997) modification of the double-Poisson model:

$$P(X = x,\, Y = y) \;=\; \tau_\rho(x, y) \cdot \mathrm{Pois}(x; \lambda_{ij}) \cdot \mathrm{Pois}(y; \mu_{ij}) \tag{1}$$

with scoring rates built from one attack strength α and one defence strength δ per team, a shared intercept c, and a shared home-advantage term h:

$$\ln \lambda_{ij} = c + h + \alpha_i - \delta_j \qquad \text{(home goals)} \tag{2}$$

$$\ln \mu_{ij} = c \qquad + \alpha_j - \delta_i \qquad \text{(away goals)} \tag{3}$$

and a low-score correction τ_ρ that nudges the four cells where goals are 0 or 1 each, because real 0–0 and 1–1 results are slightly more or less common than two independent Poisson counts would predict:

$$\tau_\rho(x,y) = \begin{cases} 1 - \lambda\mu\rho & x=0, y=0 \\ 1 + \lambda\rho & x=0, y=1 \\ 1 + \mu\rho & x=1, y=0 \\ 1 - \rho & x=1, y=1 \\ 1 & \text{otherwise.} \end{cases} \tag{4}$$

ρ is a single number shared by the whole league (typically around -0.05 to -0.10). Everything else about the joint scoreline table — 1X2, over/under, both-teams-to-score, Asian handicaps, correct score — is read off this one table $G[x,y] = P(X{=}x, Y{=}y)$ by summing the right cells; see Section 8.

5.2 Fitting: the exact objective that is maximised

Given N historical matches with results (x_m, y_m) played on dates t_m, parameters $\theta = (c, h, \{\alpha_i\}, \{\delta_i\}, \rho)$ are found by maximising a *weighted, ridge-penalised* log-likelihood:

$$\mathcal{L}(\theta) \;=\; \sum_{m=1}^{N} w_m \Big[\ln \tau_\rho(x_m, y_m) + x_m \ln \lambda_m - \lambda_m + y_m \ln \mu_m - \mu_m \Big] \;-\; \eta \sum_i \left(\alpha_i^2 + \delta_i^2\right) \tag{5}$$

where the per-match weight $w_m = \exp(-\xi \cdot \text{age}_m \text{ in days})$ down-weights old matches ($\xi = 0.0019$/day by default, an $\approx$1-year half-life — this is the standard exponential time-decay used in the football-analytics literature, e.g. Dixon & Coles themselves and later work by Constantinou & Fenton), and the ridge term with strength η shrinks a team's strengths towards the league average when it has few matches (freshly promoted teams, or teams a league hasn't played this season). Optimisation is L-BFGS-B on the negative log-likelihood, using an *analytic* gradient (implemented and unit-tested against finite differences in `tests/`, not just the numerical optimiser's own internal approximation).

5.3 How this was validated

The standard way to check a fitting routine is the same one used for any Monte Carlo or numerical method in physics: generate synthetic data from a model whose parameters are *known*, fit it, and confirm the fitted parameters recover the true ones. This project does exactly that (`sportsquant/data/synthetic.py` + the test suite): a 20-team, 5-season synthetic league is simulated by drawing scorelines directly from the Dixon–Coles grid with chosen true $(\alpha, \delta, h, \rho)$, then fit blind. Over 12 independent random synthetic leagues, the fitted home-advantage and ρ were unbiased to within ± 0.01–0.04 of the true value, and the mean absolute error on each team's attack strength was ≈ 0.045 on a scale where true strengths spanned roughly ± 0.6. The analytic gradient matched a finite-difference check to within 2×10^{-4} over 10^5-scale gradient norms.

5.4 The card / corner model

Cards and corners use the same fitting machinery (weighted, ridge-penalised, analytic-gradient Poisson regression) but with a different, non-symmetric parametrisation, since "produces cards" and "concedes cards" are conceptually different from "attacks" and "defends":

$$\ln \mathbb{E}[\text{home count}] = c + h + f_{\text{home}} - g_{\text{away}} + r_{\text{ref}} \tag{6}$$

$$\ln \mathbb{E}[\text{away count}] = c \quad + f_{\text{away}} - g_{\text{home}} + r_{\text{ref}} \tag{7}$$

with a Gaussian prior on the referee effect r (shrinking an official with few matches toward "average referee"), and the predictive distribution is a *negative binomial*, not a Poisson: $\text{Var} = \mathbb{E}\,(1 + \alpha\,\mathbb{E})$, with the extra-spread parameter α estimated from the Pearson residuals of the fit, because card and corner counts are empirically over-dispersed relative to a Poisson. **This model is only as good as the referee field in the source data**: football-data.co.uk includes it for England and Scotland; most other free European files do not, in which case the referee term is simply omitted for that match rather than guessed.

6 The in-play soccer engine: what actually moves the price after a goal

This is the direct answer to "does it refresh itself... Barcelona pays 1.25, after they score twice it goes to 1.04." Goals are modelled as two time-inhomogeneous Poisson processes whose instantaneous rate depends on three things:

1. **Time**: expected goals are spread unevenly across the match (a fitted-shape ramp per half, with 45.5% of a match's goals in the first half by default), not uniformly;
2. **Score effects**: a team leads by d goals scores at rate $\lambda \cdot e^{-\beta d}$ and the trailing team at $\mu \cdot e^{+\beta d}$ (trailing teams push, leading teams sit back) — $\beta = 0.08$ here is a literature-typical prior, not yet calibrated on this project's own data;
3. **Red cards**: a team down a man scores less and concedes more, by fixed multipliers.

Because the rate depends on the *current* score, the distribution of the final score is not a simple Poisson shift; it is stepped forward minute by minute as a genuine Markov chain (dynamic programming over the joint count of extra home/away goals). Two properties were checked directly rather than assumed:

- With $\beta = 0$ and no red cards, the in-play table reduces *exactly* to two independent Poisson counts over the remaining scoring exposure (checked to 10^{-7} against `scipy.stats.poisson`) — the dynamic-programming machinery is not doing anything beyond what a closed-form calculation would give in the case where it should not be;
- A *terminal tilt* $R(x, y) = G_{\text{DC}}(x, y)/G_{\text{DP}}(x, y)$ is applied so that at the exact moment of kickoff the in-play engine's full-match distribution equals the pre-match Dixon–Coles table *exactly* (checked to 5 decimal places) — there is no artificial jump in the odds the instant the clock starts.

The same machinery also prices "1UP/2UP" (pays out the moment a team leads by 1 or 2 goals, even if the match later draws), next-goalscorer timing, and 15-minute goal windows, each solved as an absorbing/first-passage or hazard calculation on the same minute-by-minute process — these

were cross-checked against a 20,000-path direct Monte-Carlo simulation of the same process independently coded, agreeing to within Monte-Carlo noise (0.7266 vs. 0.7252 analytically, on a check with the score-effect switched off so the comparison has a clean interpretation).

7 Sports other than soccer

Basketball / NFL — Gaussian margin model. A ridge regression on *margin* (home score − away score) and *total* (sum), each team getting one rating, solved in closed form (weighted normal equations). In-play, the remaining margin/total is treated as a Brownian motion: mean drifts by the fitted rate × time left, variance shrinks ∝ time left — the standard "win probability from score and time" construction used throughout public sports analytics.

Hockey / MLB — Poisson race. Same idea as soccer's goal model without the Dixon–Coles correlation term (rare 0–0/1–1 ties matter less at hockey/baseball scoring rates); ties are broken with one extra scored event, weighted by a fitted overtime/extra-innings home-side probability.

Tennis / volleyball — exact point-by-point Markov chains. Given a single per-point (or per-rally) win probability, the *entire* match — game, set, and match distribution, including tie-breaks — is computed exactly by dynamic programming over the point score, memoised per state (Python `lru_cache`), not simulated. Given a desired *match* win probability, the reverse problem (back out the per-point probability) is solved by one-dimensional root-finding (Brent's method). Verified: a 60% server-point-win probability reproduces the textbook 73.57% game-win probability exactly; the forward and reverse solves round-trip to 4 decimal places.

MMA / F1. MMA uses a literature-typical KO/submission/decision method-mix prior applied to a supplied win probability. F1 runs a 20,000-path Monte Carlo per race: driver rating minus a grid-penalty term, Gumbel noise (the standard choice for a "who finishes highest" ranking model, equivalent to a softmax over ratings), and an independent per-driver DNF probability.

8 From a scoreline table to every market on the slip

Every market shown — 1X2, double chance, draw-no-bet, both-teams-to-score, all total-goals lines, Asian handicaps and Asian totals (including quarter lines, settled as an equal-stake split across the two neighbouring lines), half-time/full-time, winning margin, correct score, and same-game combinations — is computed by summing probability mass over the appropriate cells of the same joint table $G[x, y]$ (`sportsquant/markets/scoredist.py`). Same-game parlay legs are priced by intersecting boolean masks over that table *jointly*, not by multiplying each leg's marginal probability — the app explicitly shows the ratio of the true joint probability to the naive product, because those two numbers are systematically different for correlated legs (e.g. "favourite wins" and "both teams score" are negatively correlated: the favourite winning big and clean-sheeting is a common way for them to win).

8.1 Odds, edge, and staking — the definitions used throughout

$$\text{decimal odds } o: \text{ a \$1 stake returns \$}o \text{ in total} \tag{8}$$

$$\text{implied probability} = 1/o \tag{9}$$

$$\text{fair odds for probability } p = 1/p \tag{10}$$

$$\text{overround} = \textstyle\sum_k 1/o_k \;-\; 1 \tag{11}$$

$$\text{edge} = p \cdot o - 1 \tag{12}$$

$$\text{Kelly fraction } f^* = \frac{p \cdot o - 1}{o - 1}, \qquad \text{staked at a quarter of } f^* \text{ by default} \tag{13}$$

Bookmaker margin is removed with Shin's (1993) method by default (also available: proportional, and the power method), which allocates the overround assuming it partly compensates for informed/insider money rather than spreading it evenly — it shades long-shot prices down more than the proportional method does, which is closer to what is empirically observed in sharp-book favourite–longshot bias.

9 What is really fitted, and what is a placeholder — read this before trusting a number

Every match in the app carries a visible confidence badge, driven directly by the code path that produced its numbers:

- **FITTED MODEL** (green): the two teams were both found in a Dixon–Coles model fit on real historical results you ingested with `scripts/ingest.py`.
- **LOW-CONFIDENCE PRIOR** (amber): one of three things happened — (a) a soccer team wasn't in your ingested history (a promoted team, a league you haven't ingested, a cup tie between two different leagues), and a league-average prior was used; (b) a non-soccer match is priced from a same-season ESPN win/loss record via a simple log-odds heuristic, because a proper historical box-score ingester for that sport is not wired up yet (Section 17 says exactly what to build); or (c) no form data was available at all, and a flat league-average prior was used.
- **DEMO MODEL** (violet): the match is one of the small set of always-on, clearly-labelled simulated exhibition matches (Section 10) that keep the app lively between real kickoffs. Its "final score" is a genuine random draw from this project's own in-play engine — so watching it is literally watching a sample path of the same model that prices real matches — but it is not a real result and never will be.

This badge is not cosmetic: it is read directly off the `confidence` field the pricing engine returns, the same field the chat is instructed to state alongside any probability.

10 The demo universe: why matches move even at 3am with nothing real on

`sportsquant/live/providers/sim_demo.py` keeps a handful of well-known fixtures (e.g. Barcelona vs. Galatasaray) cycling on a compressed clock (a full "match" takes about 8 real minutes). Each one is a single Monte-Carlo draw from the *same* in-play hazard function described in Section 6,

revealed minute by minute as the compressed clock advances, using a seed derived from the fixture name and the current time slot so the same match doesn't replay identically forever. This is how the "Barcelona shortens from 1.25 to 1.04 after two goals" behaviour can be seen within minutes of starting the app, without waiting for a real kickoff — and it is the same code path real matches use, so it is a genuine, honest demonstration of the engine rather than a scripted animation.

11 Live data and automatic refresh

A background thread polls ESPN's public (unofficial, undocumented) scoreboard JSON for every configured league every `poll_live_seconds` (15s by default) for live matches and every `poll_schedule_seconds` (10 min) for the upcoming schedule, and re-fits every soccer model every `refit_hours` once enough new results have accumulated. Markets are *not* pre-computed for every match on a timer; they are computed the moment the browser asks for a given match (a few milliseconds to about 30ms for the busiest soccer match), so the price you see is always calculated from the current score and clock, not from a stale cache. Open paper bet slips are checked against finished matches on the same loop and settled automatically once every leg's market type supports automatic settlement (1X2, handicaps, totals, BTTS, and similar; markets like same-game combos or minute windows are not auto-settled yet).

12 The chat

With no API key configured, an offline keyword router (`OfflineChat`) handles common requests — match lookups, value-bet scans, parlay pricing, Kelly sizing, de-vigging — by calling the exact same Python functions the web UI uses (`ToolBox`, shared code, not a re-implementation). With an Anthropic or Gemini key in `.env`, the same functions are exposed to the model as tools; the system prompt explicitly forbids the model from stating a number it did not get from a tool call, and this was verified structurally (not just by prompting) by mocking both providers' HTTP APIs and confirming the exact JSON the toolbox returned was what got sent back to the model on the next turn.

13 The X / Twitter layer, and why it is off by default

The project brief asked for a layer that reads soccer Twitter, including explicitly-named banter and parody accounts. `config/accounts_twitter.json` sorts accounts into four tiers (official, credentialed journalists, community/analytics, and banter/parody/unverified); only Tier 1–2 can ever produce a "fact" flag (e.g. a confirmed lineup change), and Tier 4 — which includes accounts the project brief itself identified as parody, such as parody accounts of a well-known transfer journalist — is never used for anything beyond optional sentiment colour, and `sportsquant/social/extractor.py` refuses to pass Tier 3/4 posts into fact extraction even if asked to. This module is real, working code against X's real API v2 endpoints, but it is **inactive until you add your own bearer token**: as of when this was written, X's API has no free tier for reading posts (roughly $0.005 per post read), so polling even 30 accounts a few times a day is a real ongoing cost of roughly $1–2/day and scales up from there — a decision you should make deliberately, not one this project makes for you by defaulting it on.

14 What was tested, and how

In the spirit of validating a computational model the way one would in physics — exact limits, conservation laws, and Monte Carlo cross-checks, not just "it ran without crashing":

- Every probability table produced by `ScoreDist` and by each sport's engine was checked to sum to 1 (a discrete probability conservation law) across dozens of scenarios.
- Dixon–Coles' analytic gradient was checked against finite differences; the fit was checked for parameter recovery against synthetic data with known truth, across 12 random seeds.
- The in-play engine's $\beta = 0$ limit was checked against closed-form independent Poisson counts (max abs. difference 1.0×10^{-7}); its kickoff-instant behaviour was checked to match the pre-match model exactly after the terminal tilt; its first-passage ("1UP") calculation was cross-checked against an independent 20,000-path Monte-Carlo simulation.
- The tennis Markov chain was checked against the textbook closed-form game-win probability; the round-trip (probability → per-point rate → probability) was checked to close to 4 decimal places.
- The real ESPN JSON schema was captured live and used as a test fixture after a first attempt revealed a real bug (the odds block is nested under the *competition*, not the *event* — an easy mistake the fixture test caught immediately).
- The real football-data.co.uk CSV schema (confirmed against its own published column documentation) was used to build an ingestion parser tolerant of the columns that differ by league and season.
- Every FastAPI endpoint was exercised end-to-end (event listing, market pricing, placing and auto-settling a paper bet, adding/pricing/clearing a same-match parlay, the chat, both LLM tool-calling loops with mocked HTTP).

15 What this replaces from the first (Antigravity) attempt, and why

The first pass, built directly by Google Antigravity's agents, got real things right — a Dixon–Coles core, a DuckDB store, a tiered Twitter-account concept, a working Streamlit chat – and you flagged, correctly, that it was incomplete in specific ways: the referee table in `tools.py` hard-coded four names with numbers that were not fitted from anything, and the team list only covered a handful of Premier League sides. This version fixes that class of problem structurally rather than by adding more hard-coded rows: referee and team numbers now only exist if they came out of a real fit (Section 5), and anything that has not been fitted says so on screen (Section 9) instead of presenting a guess as if it were a fact.

16 Honest limitations

- The score-effect strength β, red-card multipliers, and the weather-to-goals adjustment are literature-typical priors, not calibrated on this project's own data yet.
- Basketball, NFL, hockey and MLB do not yet have a historical box-score ingester wired up (Section 17), so they run on a same-season record heuristic until one is added.

- Referee and card/corner modelling is only as complete as the source data: most non-English free files do not include a referee column.
- The market is usually at least as well-informed as this model, and often more so. A fitted model and a positive expected value calculation are not a guarantee of anything; they are an estimate with real uncertainty around it. Treat the value-finder and Kelly-sizing tools as a paper-trading and closing-line-value research exercise, not a source of income, and re-read Section 8's point about correlated parlay legs before trusting a multi-leg number someone else's app gave you without that correction.

17 Suggested next steps

1. Run `scripts/ingest.py --all` for real Big-5-plus-extras history, then watch the confidence badges on real fixtures turn from amber to green.
2. Wire up a historical results ingester for basketball/NFL/hockey/MLB (the same `Store.add_result` table already exists; `ModelManager.refit` already fits a `MarginModel` the moment enough rows exist for a competition) — see `prompts/06_other_sports_ingestion.md`.
3. Calibrate β, the red-card multipliers, and the weather adjustment against your own ingested results instead of the literature priors currently in `InPlayParams`.
4. Only if you decide the cost is worth it: add an X bearer token and build out `extract_flags`'s LLM prompt for real injury/lineup extraction, then feed confirmed flags into the pre-match rate as an explicit multiplier with its own fitted effect size, the same way the referee effect is fitted today — not as a hard-coded adjustment.